A Review of Node Embedding Methods on Stochastic Block Models

Matthias Zelleke

September 2025

1 Introduction

1.1 Motivation

Networks[1] are a type of data found throughout the real world, with social media communities, transportation networks, and molecule structures being just a few of their many applications [31]. When combined with the growing popularity of data-driven decision-making [3, 2], which utilizes machine learning (ML) models, being able to transform information from networks into a meaningful and ready-to-use format for ML models becomes essential.

1.2 Terminology and Notation

A network $G = (V, E)$ is a data structure consisting of two sets: a vertex set V containing the *objects* of interest (e.g. people in a social network, or cities in a transportation network), and an edge set $E \subseteq V \times V$ indicating if there is a *relationship* between two elements of V (e.g. whether two people follow each other, or whether two cities are directly connected by a road). A network is called *weighted* if we assign a real-valued number to each edge, indicating the strength of its connection. A network is called *directed* if edges have an inherent direction: $(v_1, v_2) \neq (v_2, v_1)$.

Networks can be represented by their *adjacency matrix* $\mathbf{A} \in \mathbb{R}^{|V|\times|V|}$, where $\mathbf{A}[i, j] = 1$ if there is an edge between nodes i and j, and 0 otherwise. In the case of weighted networks, $\mathbf{A}[i, j]$ indicates the *weight*/strength of the edge. In the case of undirected networks, $\mathbf{A}$ is symmetric: $\mathbf{A}[i, j] = \mathbf{A}[j, i]$.

Sometimes, each node in a network also has information about it (e.g. a user's age, number of posts made, and number of likes received in a social media network). Formally, each piece of information is called a *feature*, and this information is stored in a *feature matrix* $\mathbf{X} \in \mathbb{R}^{|V|\times d}$, where each row is the feature vector for the node $u \in V$, and d is the number of features.

The type of network we will focus on is the *Stochastic Block Model* (SBM) [20]. In an SBM, nodes are grouped into a pre-specified number of groups/communities, K, and the probability of an edge between two nodes depends only on the group each node is in [20]. Thus, we can use a *block-to-block probability matrix* $\mathbf{B} \in \mathbb{R}^{K\times K}$ to describe these probabilities [20], where $\mathbf{B}[i, j]$ is the probability that there is an edge between a node in group i and a node in group j. For undirected networks, $\mathbf{B}$ is symmetric; but for directed networks it may be asymmetric: $\mathbf{B}[i, j] \neq \mathbf{B}[j, i]$

2 Classical Methods

To begin, we will investigate *Spectral Clustering* (or more generally, *Spectral Embedding*), one of the first methods used for learning node embeddings.

[1] Often called *graphs* in math and computer science

2.1 Spectral Embedding

When doing node clustering, we want nodes that are "similar" to be clustered together. "Similar" can often mean one of several things (e.g. the distance to travel between the two nodes along the network (i.e. the sum of edge weights on the shortest path between the two nodes) is small, the two nodes share a similar structure in their local neighbourhoods, they have similar node features). If we use the network-based distance between nodes as our notion of similarity, then we can think of each group of similar nodes as a cluster, as they will be more densely connected to each other than with nodes not in the cluster.

As a result, a good clustering solution should keep the number of edges across clusters small, while having the number within each cluster to be much larger. This can be formalized using the notion of a cut function [18], which is just a normalized count of the number of edges across clusters, given a partition of a network's nodes. Thus, we can state our goal as learning node embeddings, which, when clustered (using a common clustering algorithm like k-means), minimize the cut function. Note that we can interpret the output of a clustering algorithm as a partition of the nodes, since it assigns each node to one of K disjoint clusters whose union is the set of all nodes. Formally, our goal is:

$$\min RatioCut(kmeans(\mathbf{Z}, K)), \tag{1}$$

where $\mathbf{Z}$ is a matrix containing each node's embedding and *RatioCut* is the specific type of cut function we will use [18]. *RatioCut* can be computed for any partitioning/clustering of nodes, however, as we will see, it is minimized when the embeddings used to cluster the nodes are the *spectral embeddings* for the nodes in the network, which we will also define soon.

Mathematically, *RatioCut* is defined as:

$$RatioCut(A_1, ..., A_K) = \frac{1}{2} \sum_{k=1}^{K} \frac{|(u,v) \in E \ : \ u \in A_k, \ v \in V \setminus A_k|}{|A_k|}. \tag{2}$$

To minimize *RatioCut*, we will use the Laplacian matrix L of a network, which is a function of its adjacency matrix and possesses useful properties for clustering. It is defined as:

$$L = D - A, \tag{3}$$

where D is a diagonal matrix containing the node degrees, and A is the adjacency matrix. One of its useful properties we will use is [18]:

$$\mathbf{x}^{\mathbf{T}}\mathbf{L}\mathbf{x} = \sum_{(u,v)\in E} (\mathbf{x}[u] - \mathbf{x}[v])^2, \ \forall \mathbf{x} \in \mathbb{R}^{|V|}. \tag{4}$$

4 can be used to show that minimizing *RatioCut* (when there are $K = 2$ partitions) is (approximately) equivalent to solving the following optimization problem [18]:

$$\min_{\mathbf{x}\in\mathbb{R}^{|V|}} \mathbf{x}^{\mathbf{T}}\mathbf{L}\mathbf{x}, \ s.t. \ \mathbf{x} \perp \mathbb{1}, \ ||\mathbf{x}||^2 = |V|, \tag{5}$$

where $\mathbb{1}$ is the vector of all 1's. This isn't *exactly* equivalent, since a relaxation was made to allow real-valued vectors in order to simplify the problem [18].

By the Rayleigh-Ritz Theorem [14], the solution to 5 is the eigenvector corresponding to the second smallest eigenvalue of L [18]. If there are $K > 2$ partitions, the solution is given by the K smallest eigenvectors of L (excluding the smallest, i.e. the second smallest to the $(K+1)^{th}$ smallest) [18].

Putting the above points together, we can now present the Spectral Clustering algorithm [32]:

1. Compute the Laplacian matrix L of the network as $L = D - A$
2. Find the eigenvectors of L and take the K smallest (excluding the smallest)

3. Horizontally concatenate the eigenvectors into a matrix $\mathbf{Z} \in \mathbb{R}^{|V| \times K}$. The ith row of $\mathbf{Z}$ is the *spectral embedding* for node i

4. Run *kmeans* on $\mathbf{Z}$ to assign each node to its corresponding partition A_k based on the clustering result, i.e. $A_k := \{u \ : \ kmeans(\mathbf{Z}, K)[u] = C_k\} \ \forall k \in \{1, ..., K\}$; where C_k is the cluster assigned to node u

If you don't plan to use the generated embeddings $\mathbf{Z}$ for node clustering, but for a different task like node classification, then you can skip Step 4. For node classification, however, it's important to remember that this modified version of Spectral Clustering, which would be more appropriately named as *Spectral Embedding*, would only work well if the node classes form community-like structures where nodes in each class are more densely connected with themselves than with other nodes [34]. Nevertheless, there exist other variations of Spectral Embedding which overcome this limitation [11].

As Spectral Embedding requires the value of K to be specified by the user, it performs better when the number of "true clusters/classes" in the network, K_{opt} is known [1], as is the case with SBMs. However, if one doesn't know K_{opt}, and the network contains clearly defined clusters/classes, techniques like the eigengap heuristic work well in choosing K [43].

Since Spectral Embedding uses the Laplacian matrix, which is a function of the entire network, it is able to encode global information relevant to a node (i.e. which cluster it is in), into that node's embedding [43]. As for smaller, local neighbourhood information, setting $K > K_{opt}$ can reveal these smaller sub-structures [43]. However, doing so could hurt the method's ability to recover the true clusters/classes, especially if the network doesn't contain clearly defined clusters/classes [43].

3 Modern Methods

The Spectral Embedding algorithm in Section 2.1 was motivated from an attempt to minimize the number of edges between nodes in different clusters/classes. In this section, we explore more recent[2] methods that use an encoder/decoder approach and aim to reconstruct the pairwise node similarities using a decoder [18].

In the encoder/decoder approach, we encode information about each node and its local neighbourhood in the network into an embedding vector, and then try to reconstruct the node and its local neighbourhood through a decoder **DEC** which uses its embedding vector [18]. **DEC** often takes two embeddings and tries to predict how similar they are in the network, where two nodes' degree of similarity is defined using a similarity matrix $\mathbf{S}$ [18]. In contrast, the encoder **ENC** is simply a lookup table for a node's embedding: $\mathbf{ENC}(u) = \mathbf{z}_u, \ \forall u \in V$ [18]. The key to the encoder/decoder approach is that the node embeddings $\mathbf{z}_u$ are updated, using gradient descent, to minimize the difference between **DEC** and $\mathbf{S}$.

Choosing how to define **DEC** and $\mathbf{S}$ are two of the main distinctions among embedding methods that use the encoder/decoder approach [18], however, common choices for **DEC** are a dot product of embeddings $\mathbf{z}_u^T \mathbf{z}_v$ or a softmax of embedding dot products $\frac{e^{\mathbf{z}_u^T \mathbf{z}_v}}{\sum_{k \in V} e^{\mathbf{z}_u^T \mathbf{z}_k}}$; and common choices for $\mathbf{S}$ are the adjacency matrix, powers of the adjacency matrix, or transition probabilities representing the probability of two nodes co-occuring near to each other on a random walk across the network [18].

Once we have $\mathbf{S}$, the actual pairwise node similarities, and **DEC**, the predicted pairwise node similarities, we need a way of measuring the accuracy of these predictions. This is done using a loss function $\mathbf{L}$, which is the third main point of distinction among these embedding methods [18]. Depending on how **DEC** and $\mathbf{S}$ are defined, $\mathbf{L}$'s definition will differ [18]. Nevertheless, common choices include a mean-squared error (MSE) loss or a cross-entropy loss (e.g. binary cross-entropy) [18]. As mentioned above, gradient descent is used to minimize $\mathbf{L}$ by updating the node embeddings $\mathbf{z}_u$, until we obtain our final embeddings $\mathbf{Z}^*$.

[2] Recent relative to Spectral Embedding (2002) [32], as the earliest of these methods was published in 2015 [5, 39]

3.1 Node2vec

Node2vec [15] is a node embedding algorithm inspired by the word2vec [30] algorithm used in natural language processing (NLP). Just as word2vec uses sentences of words to learn which words occur close to each other in sentences [30], node2vec uses random walks across a network to learn which nodes appear close to each other along such random walks [15].

A user-specified number of random walks is generated starting from each node. To determine which node to visit next in a walk, two parameters p and q are used. If $p = 1$ and $q = 1$, the random walk is unbiased, meaning the probability of moving to an adjacent node is proportional to the weight of the edge between the current node and that adjacent node [15]. In fact, setting $p = 1$ and $q = 1$ makes node2vec equivalent to DeepWalk [34], another random-walk-based node embedding algorithm.

In the original paper, the node2vec authors call p the *return parameter*, since setting $p > \max(q, 1)$ decreases the chance of re-visiting the node you just visited in the walk (i.e. taking a step back), and setting $p < \min(q, 1)$ increases that chance [15]. q is called the *in-out* parameter since setting $q > 1$ makes the random walk more likely to stay around the start node, and $q < 1$ makes the random walk biased towards moving away from the start node [15].

For a pair of nodes (u, v), the similarity measure $\mathbf{S}[u, v]$ used by node2vec is the proportion of times v appears within a distance w of u across all the random walks generated for the network [15]. In this setup, v is called a *context node*, u is called the *centre node*, and w is the *window size* [15]. In symbols [15]:

$$\mathbf{S}[u, v] = \frac{|\#\ times\ v\ is\ a\ context\ node\ when\ u\ is\ the\ centre\ node|}{|\#\ times\ u\ is\ the\ centre\ node|} = p(v|u) \tag{6}$$

where we use $p(v|u)$ to highlight that the values of $\mathbf{S}$ represent conditional probabilities.

Node2vec uses a softmax-based decoder which aims to estimate $\mathbf{S}[u, v]$ [15]. Formally, the decoder is [15]:

$$\mathbf{DEC}(\mathbf{z}_u, \mathbf{z}_v) = \frac{e^{\mathbf{z}_u^T \mathbf{z}_v}}{\sum_{k \in V} e^{\mathbf{z}_u^T \mathbf{z}_k}} = \hat{p}(v|u) \tag{7}$$

where we use $\hat{p}(v|u)$ to highlight that the decoder's output represents an estimate of the conditional probability of being at node v within a distance of w from node u, along a random walk.

An issue with this decoder is that the denominator requires iterating over all nodes in the network for *each pair*, which increases the compute time [15]. To fix this, we can use negative sampling [30], a noise contrastive approach [17] allowing us to randomly sample nodes k_i from the network so that each positive pair (u, v) has Q negative pairs $(u, k_1), ..., (u, k_Q)$. which node2vec interprets as "u and v should be similar since they co-occur, whereas u and k_i shouldn't be since they don't co-occur as often". Negative sampling is noise contrastive since it allows us to simulate noisy/fake samples which can later be contrasted with real samples in a loss function. The form of negative sampling used by node2vec isn't uniform, but is adjusted so that a node's probability of being sampled as a negative node is proportional to its degree raised to the $\frac{3}{4}$ power: $P_{neg}(u) = \frac{d_u^{3/4}}{\sum_{v \in V} d_v^{3/4}}$ [30]. This is done so that nodes with higher degrees are more likely to be negative nodes (since they appear in general along the walks more often) while ensuring they don't over-dominate, allowing lower degree nodes to get sampled as well [13].

Since node2vec aims to learn node embeddings which encode whether a pair of nodes (u, v) co-occurred or not, it uses a binary cross-entropy (BCE) loss, enabling it to learn to differentiate/contrast the "positive pairs" (the aforementioned pairs which co-occurred) from the "negative pairs" (the negative pairs (u, k_i) produced by negative sampling) [15]:

$$\mathbf{L} = \sum_{(u,v) \in D} \left(-\log(\sigma(\mathbf{z}_u^T \mathbf{z}_v)) - \sum_{i=1}^{Q} \log(1 - \sigma(\mathbf{z}_u^T \mathbf{z}_{k_i}))\right) \tag{8}$$

where D is the *multiset* of all pairs (u, v) which co-occurred, $\sigma(x) = \frac{1}{1+e^{-x}}$ is the sigmoid function, and the negative samples k_i are independently drawn for each $(u, v) \in D$. This loss $\mathbf{L}$ is minimized by updating the embeddings $\mathbf{z}_u, \forall u \in V$ using stochastic gradient descent (SGD) (or one of its variants) [15]. This paper's accompanying node2vec code (which fully uses the *node2vec* package by eliorc) [8], uses Gensim's word2vec optimizer [30], which uses SGD with a linearly decaying learning rate, by default.

3.2 Large-scale information network embeddings (LINE)

Large-scale information network embeddings (LINE) [39] is an embedding method which considers two goals when learning node embeddings:

LINE uses a normalized version of the adjacency matrix as the similarity matrix for the *first-order proximity*: $\mathbf{S}_1[u, v] = \frac{\mathbf{A}[u,v]}{\sum_{(u',v')\in V^2} \mathbf{A}[u',v']}$ [39], which is essentially the proportion which the edge (u, v)'s weight contributes to the total edge weight across the network. The similarity matrix for the *second-order proximity* is the probability distribution over a node u's neighbours [39]: $\mathbf{S}_2[u, v] = \frac{\mathbf{A}[u,v]}{\sum_{v'\in V} \mathbf{A}[u,v']} = p_u(v)$, which is essentially the proportion which the edge (u, v)'s weight contributes to u's degree[3].

For the pairwise decoder, the *first-order* component uses the sigmoid of the dot product between two node embeddings: $\mathbf{DEC}_1(\mathbf{z}_u, \mathbf{z_v}) = \sigma(\mathbf{z}_u^T \mathbf{z}_v)$ [39], and the *second-order* component uses the same decoder as node2vec in Section 3.1, with the slight subtlety that each node has a centre embedding $\mathbf{z}_u$ and a context embedding $\mathbf{z}'_u$. The former is used when u is the left-hand side (i.e. source) node, and the latter when u is the right-hand side (i.e. destination) node or a negative sample node. [39]. To make this clear, we can write the *second-order* decoder as [39]:

$$\mathbf{DEC}_2(\mathbf{z}_u, \mathbf{z}_v) = \frac{e^{\mathbf{z}_u^T \mathbf{z}'_v}}{\sum_{k\in V} e^{\mathbf{z}_u^T \mathbf{z}'_k}} = \hat{p}_u(v) \tag{9}$$

where u is the centre node and the v and k's are the context nodes. As with node2vec, LINE employs negative sampling, also using $P_{neg}(u) = \frac{d_u^{3/4}}{\sum_{v\in V} d_v^{3/4}}$ to avoid the extra cost associated with computing the denominator. [39].

As mentioned earlier, a decoder aims to reconstruct the similarity matrix, thus, LINE's authors try to minimize the *distance* between the empirical (similarity matrix) and predicted (decoder output) probability distributions [39]. They use the KL-divergence as the measure of *distance* [39], allowing the loss functions for the *first-order* and *second-order* components, respectively, to be written as [39]:

$$\mathbf{L}_1 = \sum_{(u,v)\in E} -\mathbf{A}[u, v] \log(\sigma(\mathbf{z}_u^T \mathbf{z}_v)) \tag{10}$$

$$\mathbf{L}_2 = \sum_{(u,v)\in E} (-\mathbf{A}[u, v] \log(\sigma(\mathbf{z}_u^T \mathbf{z}'_v)) - \sum_{i=1}^{Q} \mathbf{A}[u, v] \log(1 - \sigma(\mathbf{z}_u^T \mathbf{z}'_{k_i}))) \tag{11}$$

where, again, $\mathbf{z}'_u$ is the context embedding for a node u, σ is the sigmoid function, and the negative samples k_i are independently drawn for each edge (u, v). The authors' implementation uses asynchronous stochastic gradient descent (ASGD) as the optimizer; however, the optimizer used in this paper's accompanying LINE implementation (which was written by dmpierre) [10], uses Adam, a popular variant of SGD that uses statistics about the gradient to enable faster and more stable convergence [24].

The final LINE node embeddings are generated by concatenating the node embeddings obtained after optimizing the *first-order* and *second-order* loss functions $\mathbf{L}_1$ and $\mathbf{L}_2$, respectively. [39]. Note that for the *second-order* proximity, it is the centre embeddings $\mathbf{z}_u$ that should be concatenated. These final embeddings are very good at encoding local neighbourhood information, which isn't surprising given the definition of the *first-order* and *second-order* proximities. However, LINE is

[3] In the case of a directed network, it would be the proportion which the edge (u, v) contributes to u's out-degree

also able to indirectly encode global structural information from the network into node embeddings, due to the overlapping of neighbourhoods across the network, allowing structural information to be propogated between nodes which may not have any neighbours in common [4]. Nevertheless, as other methods like node2vec explicitly capture global network structure, they are probably better equipped for dealing with large networks that contain complex structures and several global structures.

3.3 GraRep

In contrast to node2vec and LINE, GraRep [5] is a fully deterministic[4] node embedding algorithm, as it does not use random walks, negative sampling, or other forms of non-determinism. Moreover, GraRep doesn't use a decoder, a loss function, or any optimization.

At a high level, GraRep aims to learn node embeddings which capture the entire global structure of the network by performing dimension reduction on various powers of the row-normalized adjacency matrix that get further transformed (to control the influence of certain nodes). Note that the kth power of the adjacency matrix contains information about the connectivity of nodes within a distance of k edges of each other. More precisely:

$$\mathbf{A}^k[u, v] = \# \text{ of walks of length } k \text{ between nodes } u \text{ and } v \tag{12}$$

In a sense, the powers of the adjacency matrix could be viewed as different similarity matrices $\mathbf{S}$ for GraRep.

The set of nodes within a distance of k edges of a given node is also called its k-hop neighbourhood, and it is closely related to the idea of a kth *order proximity* [16], revealing that GraRep can capture more global structure than LINE (when $K > 2$), while doing so explicitly.

Here is the full GraRep algorithm in detail [5]:

1. Obtain the adjacency matrix $\mathbf{A}$ of the network.
2. Compute the row-normalized adjacency matrix $\mathbf{P}^1 = \mathbf{D}^{-1}\mathbf{A}$, where $\mathbf{D} \in \mathbb{R}^{|V|\times|V|}$ is a diagonal matrix with node degrees on the main diagonal (called the *degree matrix* of the network). $\mathbf{P}^1[u, v]$ is the probability of moving to node v given you are currently at node u and use the weights of the edges touching u to determine which node to go to next (i.e. the probability is positively proportional to the edge weight). $\mathbf{P}^1$ is also called the 1-step transition probability matrix.
3. For each $k \in \{1, ..., K\}$:
   (a) Compute the k-step transition probability matrix $\mathbf{P}^k$.
   (b) Compute the column-wise sums $\{\Gamma_j^k\}_{j=1..|V|}$ of $\mathbf{P}^k$ as $\Gamma_j^k = \sum_p \mathbf{A}[p, j]$. These sums can be interpreted as being proportional to how likely a node j would be to appear if a random walk were to be done [35].
   (c) Compute the log probability matrix $\mathbf{X}^k$ as[5]: $\mathbf{X}^k[i, j] = \log(\frac{\mathbf{P}^k[i,j]}{\Gamma_j^k}) + \log(|V|)$. $\mathbf{X}^k$ can be interpreted as a transformed version of $\mathbf{P}^k$ that empahasizes node pairs (i, j) which co-occur more often than expected given the general frequency with which j already appears [7, 5].
   (d) Set any negative entries in $\mathbf{X}^k$ to 0 to reduce noise [5].
   (e) Compute the Singular Value Decomposition (SVD) of the log probability matrix $\mathbf{X}^k$: $[\mathbf{U}^k, \mathbf{\Sigma}^k, (\mathbf{V}^k)^T] = SVD(\mathbf{X}^k)$. The rows of $\mathbf{U}^k(\mathbf{\Sigma}^k)^{\frac{1}{2}}$ and $\mathbf{V}^k(\mathbf{\Sigma}^k)^{\frac{1}{2}}$ give us row embeddings and column embeddings of $\mathbf{X}^k$, respectively [26, 27]. Since the row embeddings alone capture enough information regarding a node's relationship to its k-hop neighbourhood, GraRep discards the column embeddings [5].

[4] Certain SVD solvers like randomized SVD solvers are non-deterministic

[5] The original paper specifies $-\log(\beta)$ as the constant term added, however, both the authors' and this paper's implementation set $\beta = \frac{1}{|V|}$, resulting in $\log(|V|)$

(f) Set the k-step (lower dimensional) row embeddings $\mathbf{W}^k \in \mathbb{R}^{|V| \times d}$ as $\mathbf{W}^k = \mathbf{U}_d^k (\mathbf{\Sigma}_d^k)^{\frac{1}{2}}$, where the subscript d indicates that the first d entries are kept, allowing us to obtain lower dimensional row embeddings.

4. Concatenate the k-step lower dimensional row embeddings into the final embedding matrix: $\mathbf{W} = Concat(\mathbf{W}^1, ..., \mathbf{W}^k) \in \mathbb{R}^{|V| \times dK}$

By combining the embeddings for each k, GraRep is able to encode both local and global structure into a node's embedding.

4 Graph Neural Networks

Graph Neural Networks (GNN) are a more recent class of network embedding methods grounded in the concept of deep learning [29], enabling the use of neural networks and backpropagation to manipulate information about the network's structure (contained in the adjacency matrix $\mathbf{A}$) and any node features (contained in a feature matrix $\mathbf{X}$) [44].

The architecture of GNNs is centred around the message-passing framework[6] [12, 18], which is an iterative framework describing how GNNs propogate information from one node to other nodes [18]. The message-passing framework has two core components/functions, which are applied at each iteration [18]:

- An aggregator function $AGGREGATE^{(k)}$ which takes in information, in the form of hidden embeddings $\mathbf{h}_u^{(k)}$, from a node u's neighbourhood $N(u)$ and outputs an aggregated embedding $\mathbf{m}_{N(u)}^{(k)}$
- An updater function $UPDATE^{(k)}$ which takes in the aggregated embedding $\mathbf{m}_{N(u)}^{(k)}$ from a node's neighbourhood and uses it to update the node's hidden embedding at the current iteration $\mathbf{h}_u^{(k)}$, outputting the node's hidden embedding for the next iteration $\mathbf{h}_u^{(k+1)}$

The above list can be succinctly captured by the following update equation for GNNs at each iteration k [18]:

$$\mathbf{h}_u^{(k+1)} = UPDATE^{(k)}(\mathbf{h}_u^{(k)}, AGGREGATE^{(k)}(\{\mathbf{h}_v^{(k)}, \forall v \in N(u)\})), \tag{13}$$

where $\mathbf{h}_u^{(0)} = \mathbf{X}[u]$ are used as the initial (base case) embeddings, and where $AGGREGATE$ and $UPDATE$ can be any differentiable function, including neural networks [18]. The superscripts indicate that the hidden embeddings can be different at each iteration and that the two core functions have different sets of parameters for each iteration.

In practice, $K = 2$ or $K = 3$ iterations gives the best performance with GNNs [25], as many GNNs, such as Deep Graph Infomax [42], include additional functions which aggregate information from the entire network, and since node features can often carry global network information as well [44]. As a result, GNNs are well-equipped to capture both structural and feature-based information into node embeddings.

We will now explore two common GNNs, GraphSage and Deep Graph Infomax, in detail.

4.1 GraphSage

GraphSage is a node embedding method that learns a function (a neural network) for generating node embeddings [19]. Unlike many GNNs, GraphSage is also *inductive*, meaning that it can generate embeddings for new nodes not seen during training, and even whole new networks [19].

As it also follows the message-passing framework, GraphSage uses an $AGGREGATE$ and $UPDATE$ function [19]. The original authors specify three options for the aggregator: a plain

[6] There are some GNNs which aren't based on this framework, such as ChebNet [9] and Graph Transformer Networks [45]

element-wise mean aggregator, an LSTM aggregator, and a pooling aggregator [19]. This paper's implementation uses a *convolutional*[7] version of the mean aggregator proposed by the authors [19], as it showed improved classification performance on the SBM compared to the pooling aggregator, and was fairly simple to implement. Mathematically, this convolutional mean aggregator is:

$$AGGREGATE^{(k)} = ReLU(\mathbf{W}^{(k)} MEAN(\{\mathbf{h}_u^{(k)}\} \cup \{\mathbf{h}_v^{(k)}, v \in N(u)\})) \tag{14}$$

where $ReLU(x) = \max(0, x)$ is the rectified linear unit, and where $MEAN$ denotes the element-wise mean. As this aggregator includes the current node in the aggregation, the $UPDATE$ function can be removed when using it [19]. Nevertheless, GraphSage's updater function is [19]:

$$UPDATE^{(k)} = \sigma(\mathbf{W}^{(k)} CONCAT(\mathbf{h}_u^{(k)}, \mathbf{m}_{N(u)}^{(k)})) \tag{15}$$

where σ can be any non-linear activation function. The authors use $ReLU$ for the activation function in their implementation. At the end of every iteration, all the current hidden embeddings are normalized to avoid numerical instability and allow gradient descent to converge faster [19, 18].

In order to control the memory and runtime, GraphSage doesn't actually use a node's entire neighbourhood, but instead uniformly samples a fixed number of neighbours, for each iteration [19].

To learn the parameters in the aggregator and updater functions, GraphSage uses a BCE loss function based on random walks [19], which is also the same as node2vec's loss function. GraphSage also uses the same strategy as node2vec (assuming $p = q = 1$ in node2vec) for generating its random walks [19], meaning that the probability of going to a specific node in the next step is proportional to the edge weight between the current node and that node. However, the authors note that this loss function can be replaced or augmented by an appropriate task-specific loss function if the node embeddings will be used on a specific downstream task (e.g. using a mean-squared error loss for a linear regression task) [19].

This paper's implementation of GraphSage also uses mini-batch sampling to make the algorithm scalable to larger networks, and to ensure that the sampled neighbourhoods for a given batch can fit in memory.

4.2 Deep Graph Infomax (DGI)

Deep Graph Infomax (DGI), another GNN adhering to the message-passing framework, was developed using ideas from information theory [42]. More precisely, DGI learns node embeddings which maximize the mutual information, based on the Jensen-Shannon divergence (JSD) [28], between themselves and a global network summary vector $\mathbf{s}$ [42]. JSD maximizes mutual information using a noise-contrastive approach [28], and DGI generates the required negative samples using a corruption function $\mathcal{C}(\mathbf{X}, \mathbf{A})$ which corrupts/alters the network's adjacency matrix and feature matrix [42]. When DGI is trained *transductively*[8] on only one network, the corruption function used keeps the adjacency matrix the same, but shuffles the rows of the feature matrix, having the effect of permuting the node IDs/labels in the network [42].

The three main components used to generate the node embeddings are the encoder, the readout function, and the discriminator function.

The encoder $\mathcal{E}(\mathbf{X}, \mathbf{A})$ is a neural network responsible for generating the node embeddings [42], and its weight matrices are learned during the loss function's optimization [42]. The authors specify slightly different choices for the encoder, depending on whether DGI is applied on *inductively* or *transductively* and whether it is applied to a single network or multiple networks [42]. However, they also state that, in theory, DGI can work with any encoder (e.g. Graph Convolutional Networks (GCN), GraphSage, Graph Attention Networks (GAT)) [42]. These three types of encoders differ mainly in how they give importance/weight to each node's neighbours when they aggregate

[7] The authors call it convolutional because it approximates a localized spectral convolution

[8] Transductive learning is the opposite of inductive, that is, when the learned model can only be applied to nodes seen during training

information from them[9] [44]. This paper's implementation uses the encoder for transductive learning on a single network (which is a standard GCN encoder), which, at each layer, does the following update [42]:

$$\mathbf{H}^{(k+1)} = ReLU(\tilde{\mathbf{A}}\mathbf{H}^{(k)}\mathbf{W}^{(k)}), \tag{16}$$

where $\mathbf{H}^{(0)} = \mathbf{X}$ are the initial node embeddings, $\tilde{\mathbf{A}} = \mathbf{D}^{-\frac{1}{2}}(\mathbf{A}+\mathbf{I})\mathbf{D}^{-\frac{1}{2}}$ is the symmetric normalized adjacency matrix with self-loops, $\mathbf{H}^{(k)} \in \mathbb{R}^{|V|\times d}$ is the node embedding matrix at iteration k, and $\mathbf{W}^{(k)}$ is the learnable weight matrix, at iteration k, for the neural network. More concretely, $\tilde{\mathbf{A}}$ is the adjacency matrix (with edge-loops for each node) normalized so that the entries reflect the relative magnitude of each edge weight after accounting for the degrees of both nodes [25]. The inclusion of self-loops in the computation allows the $AGGREGATE$ and $UPDATE$ steps of the message-passing framework to be done simultaneously, similar to GraphSage's convolutional mean aggregator.

The readout function $\mathcal{R}(\mathbf{H})$ takes in the final embeddings for *each* node and aggregates[10] them to produce the global network summary vector $\mathbf{s}$ [42]. As we will be maximizing the mutual information between the final embeddings and this summary vector, each node's embedding will be able to encode global information from the network, both structural and feature-based [42]. The specific readout function used by the authors and this paper's implementation is the sigmoid of the element-wise mean of all the final node embeddings [42]:

$$\mathbf{s} = \mathcal{R}(\mathbf{h}_1, ..., \mathbf{h}_{|V|}) = \sigma\left(\frac{1}{|V|}\sum_{i=1}^{|V|}\mathbf{h}_i\right). \tag{17}$$

The discriminator function $\mathcal{D}(\mathbf{h}, \mathbf{s})$ computes the probability that a node embedding $\mathbf{h}$ (which may have been generated from the real network $(\mathbf{X}, \mathbf{A})$ $(\mathbf{h} = \mathbf{h}_i)$ or from its corrupted version $(\tilde{\mathbf{X}}, \tilde{\mathbf{A}})$ $(\mathbf{h} = \tilde{\mathbf{h}}_i)$) is for a node from the real network [42]. It does this by comparing the embedding with the summary vector $\mathbf{s}$, as $\mathbf{s}$ was produced from the *real* network [42]. Precisely [42]:

$$\mathcal{D}(\mathbf{h}, \mathbf{s}) = \sigma\left(\mathbf{h}^T\mathbf{W}\mathbf{s}\right), \tag{18}$$

where σ is the sigmoid function, and $\mathbf{W}$ is a learnable weight matrix that's tuned (during backpropagation) so that $\mathcal{D}$ can properly distinguish real node embeddings from corrupted ones [42]. Note that $\tilde{\mathbf{h}}_i$ is generated from the *same* learned encoder $\mathcal{E}$ (that had its weights updated using the real network embeddings $\mathbf{h}_i$) [42]. The only difference is that this encoder is now run on the corrupted network: $\tilde{\mathbf{h}}_i = \mathcal{E}(\tilde{\mathbf{X}}, \tilde{\mathbf{A}})_i$ [42].

As with GraphSage, DGI employs a noise-contrastive loss function using a BCE loss function with the $\mathcal{D}(\mathbf{h}_i, \mathbf{s})$ scores being the "positive samples" and the $\mathcal{D}(\tilde{\mathbf{h}}_i, \mathbf{s})$ scores being the "negative samples" [42]:

$$\mathbf{L} = -\frac{1}{2|V|}\sum_{i=1}^{|V|}\log\left(\mathcal{D}(\mathbf{h}_i, \mathbf{s})\right) + \log\left(1 - \mathcal{D}(\tilde{\mathbf{h}}_i, \mathbf{s})\right), \tag{19}$$

where $\tilde{\mathbf{h}}_i$ are the final embeddings for the nodes from the corrupted network. Minimizing $\mathbf{L}$ maximizes the mutual information using the JSD criterion [42]. During backpropagation, this loss is minimized with respect to the weights in $\mathcal{E}$ and $\mathcal{D}$ [42]. After training is complete, one final forward pass through the encoder $\mathcal{E}$ is done to obtain the final node embeddings $\mathbf{h}_u, \forall u \in V$ [42].

As $\mathbf{s}$ is a function of *all* the nodes in the network [42], we can not use mini-batching during optimization. However, there exist potential workarounds to this, including using a cluster/subgraph-based summary vector for the nodes in a given batch [6], although this prevents $\mathbf{s}$ from being a truly "global" summary. Nevertheless, both the authors' and this paper's implementation use full-batch gradient descent with the Adam optimizer.

[9] GCNs emphasize edges which have large weights relative to their degrees [42], GraphSage uses a hand-picked approach (e.g. mean, LSTM, max-pooling) [19], and GATs use a self-attention mechanism [41]

[10] Not to be confused with the local neighbourhood aggregator function $AGGREGATE$

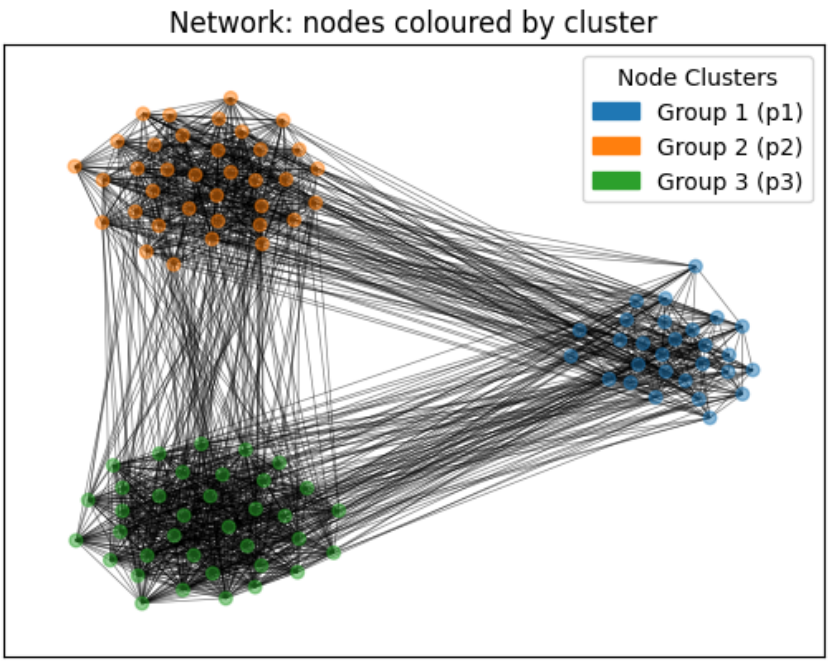


Figure 1: A visualization of the SBM, with nodes coloured and labelled according to their cluster

5 Experiments

We used a randomly generated SBM with 3 clusters and 100 total nodes to compare the performance of the aforementioned node embedding methods on a node clustering and a node classification task.

5.1 Dataset

The SBM was generated using the following block-to-block probability matrix:

$$\mathbf{B} = \begin{bmatrix} p_1 & 0.1 & 0.1 \\ 0.1 & p_2 & 0.1 \\ 0.1 & 0.1 & p_3 \end{bmatrix}, \tag{20}$$

where p_1, p_2, p_3 are adjustable parameters that control the probability of an edge existing between two nodes in the same cluster. For our SBM, we used $p_1 = 0.5$, $p_2 = 0.7$, and $p_3 = 0.9$ so that the within-cluster densities were different for each cluster while still being larger than the between-cluster density ($= 0.1$). Figure 1 shows a plot of this SBM network.

5.2 Node Clustering

5.2.1 Performance

To assess each method's performance, we applied the k-means clustering algorithm (with $k = 3$) on the *node embeddings* that the method generated, to obtain the predicted clusters. We then measured the similarity between these predicted node clusters with the actual node clusters using the normalized mutual info (NMI) score. This procedure was repeated 20 times for each embedding method, and each repetition used a different SBM network (due to the inherent stochasticity of SBMs), however, all embedding methods used the same SBM network within a repetition. Table 1 shows the mean and standard deviation of the NMI scores for each method.

Given two clusterings/partitionings of a set of nodes, the NMI score returns a value in $[0, 1]$ [38], which is a normalization of the amount of mutual information between the clusterings [37]. One can interpret the NMI score as the percentage of uncertainty in the actual clustering assignment lost when the predicted clustering assignment is known. In simpler terms, the NMI score measures how similar two clustering assignments are, with 0 meaning the two don't share any information,

and 1 meaning they perfectly correlate (i.e. identical clusterings, up to a permutation of the labels[11]) [38]. Note that there is no direct relationship between accuracy and NMI score. For example, an NMI of 0.5 *does not* mean that 50% of the points/nodes were classified correctly [38].

According to Table 1, all the embedding methods generated embeddings which, when clustered using k-means, recovered the true clusters in the SBM network quite well. This isn't surprising since the three clusters in the network are clearly separated, meaning there are more edges within each cluster than between different clusters. As this information is stored in the adjacency matrix $\mathbf{A}$, the embedding methods can easily distinguish between nodes in different clusters.

Table 1: Performance of Node Embedding Methods on Node Clustering

Node Embedding Method	NMI Score (mean ± std)
Spectral Embedding	0.99 ± 0.02
Node2vec	**1.00 ± 0.00**
LINE	0.87 ± 0.04
GraRep	**1.00 ± 0.00**
GraphSage	0.75 ± 0.09
DGI	0.93 ± 0.07

5.2.2 Embeddings and K-means Visualization

Figure 2 shows, for each embedding method, all 100 embeddings plotted using their top two principal components[12]. Each embedding/point is coloured according to its actual cluster, and the background is shaded to represent the cluster that would be predicted by k-means at that point[13].

The plots in Figure 2 show that each embedding method (other than GraphSage) produced fairly distinct clusters, with Node2vec and GraRep producing the most distinct embeddings when visualized in this two-dimensional space. Since the cluster boundaries were generated from *only* the top two principal components of the embeddings (rather than the full embeddings), some information is lost from their original embeddings, making it hard for k-means to recover the true clusters. This can be seen in Figures 2a and 2f where it seems like the embeddings generated by Spectral Embedding and DGI were not properly separated, but their high NMI scores in Table 1 show that they were properly separated in the full-dimension embedding space.

5.3 Node Classification

5.3.1 Performance

For node classification, we used a multi-class logistic regression (MLR) classifier[14] to evaluate the different methods' node embeddings. Like (binary) logistic regression, MLR uses hyperplanes in the feature space to define the boundaries between different classes. If there are C classes, it produces $C - 1$[15] hyperplanes. We randomly sampled 80% of the network's nodes to be our training set and trained an MLR classifier on them. We then evaluated the classifier's performance on the other 20% (the test set) using the following performance metrics: *accuracy, (macro) precision, (macro) recall, F1-score, (macro) F1-score.* Table 2 shows the obtained metrics.

The F1-score is the harmonic mean of the precision and recall [40]. Simply put, it is a modified mean of precision and recall that is more penalizing if one of them (precision or recall) is much lower than the other. This is useful because it lets us detect if the classifier is over-predicting a

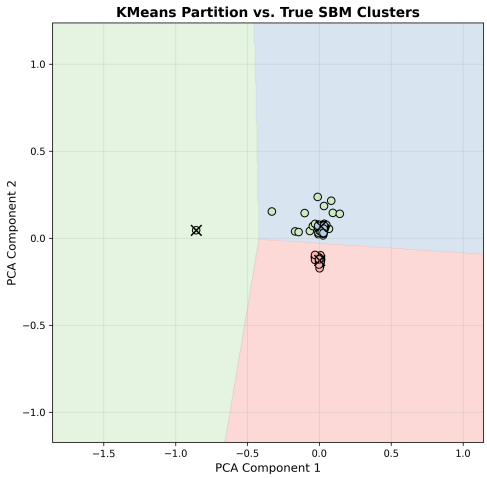
[11] When clustering, we don't care about the ordering of the cluster labels. Which cluster is "Cluster 1" and which is "Cluster 2" isn't important; the elements/nodes they contain are what matter

[12] The top k *principal components* of a set of points in some d-dimensional co-ordinate space ($d > k$) are the k directions in that space which capture the most spread/variance in the points [33]

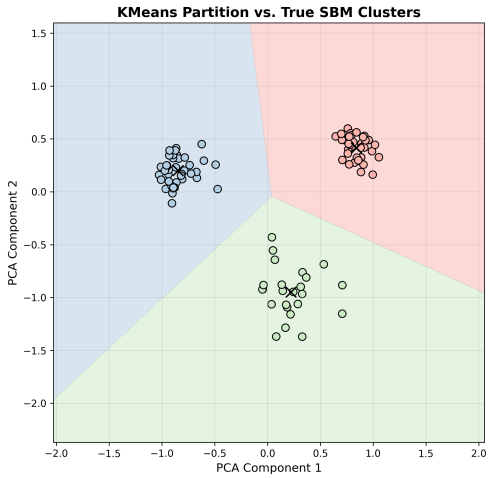
[13] The colours for the points on the plot and the background were aligned to make the visualization look nicer

[14] Multi-class logistic regression is just an extension of (binary) logistic regression to more than 2 classes [21]

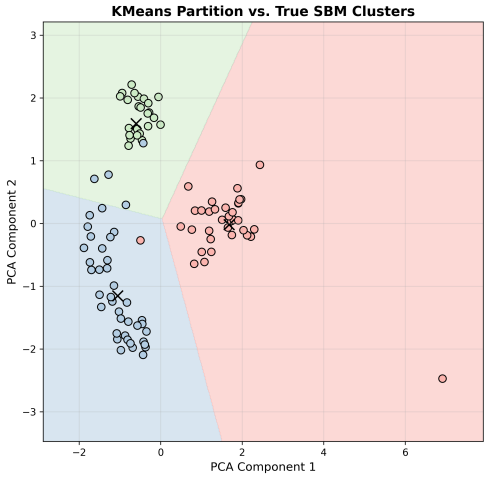
[15] Assuming the softmax version/formulation of MLR is used



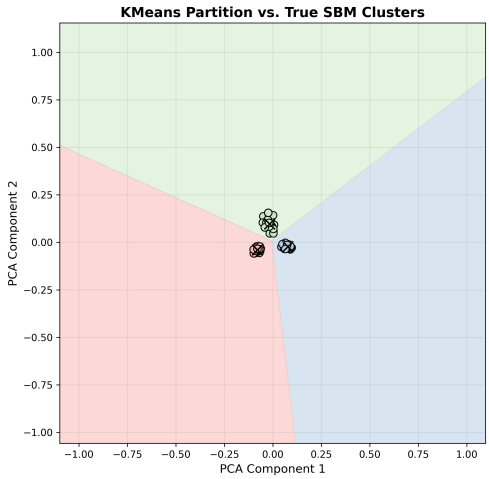
(a) Spectral Embedding



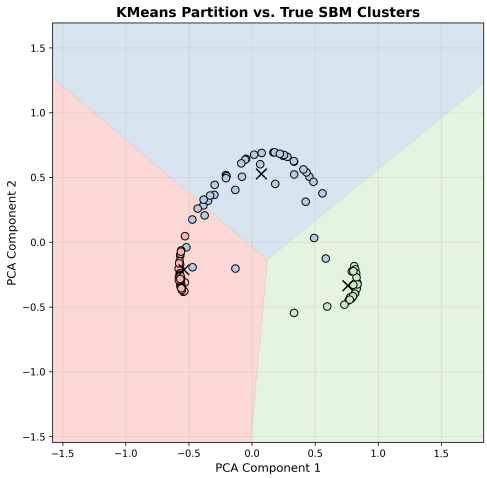
(b) Node2vec



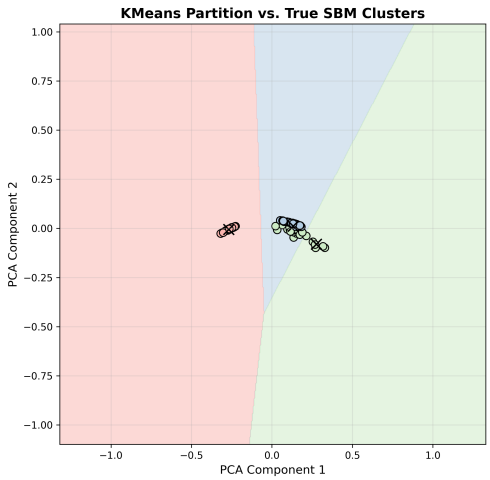
(c) LINE



(d) GraRep



(e) GraphSage



(f) DGI

Figure 2: Plots of the top two principal components for the embeddings produced by each embedding method. The points (embeddings) are coloured by their *predicted* cluster, obtained from the k-means clustering algorithm.

class[16] (low precision), or having difficulty identifying data points from a given class (low recall). The macro F1-score is simply the arithmetic mean of each class' F1-score.

Based on the results in Table 2, the MLR classifier was able to successfully classify nodes using their generated embeddings (except for the node2vec embeddings). This isn't surprising since the three classes in the network are clearly separated, allowing the pairwise adjacency information encoded in the adjacency matrix $\mathbf{A}$ to be sufficient for achieving good performance. For the same reasons as mentioned in the node clustering section, node2vec performed poorly, and only reached 60% accuracy.

Table 2: Performance of Node Embedding Methods on Node Classification

Node Embedding Method	Accuracy	Macro F1-score	Macro Precision	Macro Recall
Spectral Embedding	0.99 ± 0.02	0.99 ± 0.02	0.99	0.99
Node2vec	0.99 ± 0.01	0.99 ± 0.01	0.99	0.99
LINE	0.99 ± 0.03	0.99 ± 0.03	0.99	0.99
GraRep	**1.00 ± 0.00**	**1.00 ± 0.00**	**1.00**	**1.00**
GraphSage	0.95 ± 0.05	0.94 ± 0.06	0.95	0.95
DGI	0.99 ± 0.02	0.99 ± 0.02	0.99	0.99

5.3.2 Logistic Regression Visualization

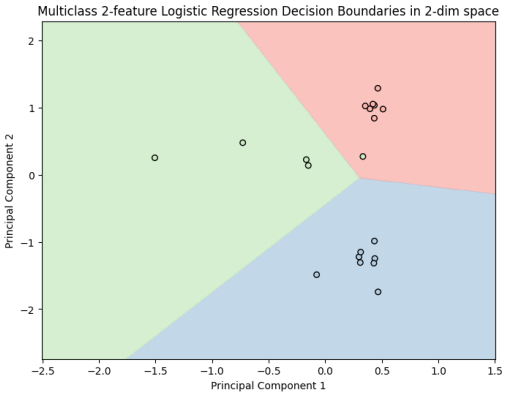
Figure 3 shows, for each embedding method, the test set embeddings plotted using their top two principal components. Each embedding/point is coloured according to its actual class, and the background is shaded according to the predicted class. Figure 3 didn't use the same MLR classifier used to obtain the performance metrics, but a separate one which took the top two principal components of the embeddings (rather than the full embeddings) as input. This was done because the original MLR took in 32-dimensional embeddings as input (or 30 and 3, in the case of GraRep and Spectral Embedding, respectively) and the background of the plot only had 2 dimensions. On the other hand, some information from the original higher-dimension embeddings is lost, as all of the variation in the higher-dimension space can't be captured using two dimensions [22]. This drawback can sometimes cause inconsistencies between performance metrics and the plots [23]. For example, GraRep attained 100% accuracy on the test set, but its plot in Figure 3 seems to show a node in the green class mispredicted as being part of the blue class.

6 Summary

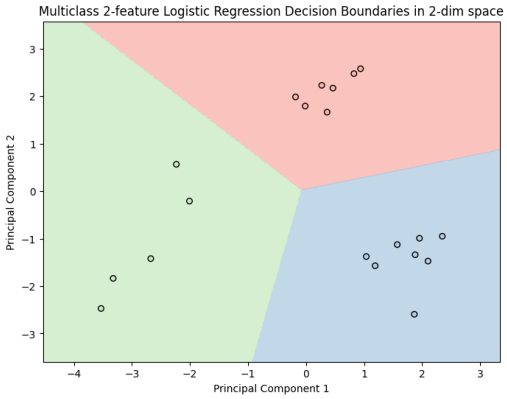
In this paper, we reviewed different embedding methods for representing nodes in a network using real-valued vectors. We considered a classic approach, Spectral Embedding [32], which uses the eigenvectors of the network Laplacian; modern methods, like node2vec and LINE [15, 39], which use the more general notion of a similarity matrix $\mathbf{S}$ along with a loss function, and like GraRep [5], which computes the SVD on powers of the adjacency matrix; and neural network-based methods, such as GraphSage and DGI [19, 42], which utilize the message-passing framework to propagate information through the network (though, in practice, only 2-4 layers of propagation are used [36]).

We also explored each method's performance on a SBM network with 100 nodes and 3 clusters/classes using a node clustering and node classification task. Each method performed very well on each task, with the exception of GraphSage for node clustering, which still performed fairly well (NMI=0.75). Each method achieved an NMI score close to or above 0.9 for node clustering, and an accuracy and F1-score above or equal to 0.95. GraRep and node2vec obtained perfect scores for the node clustering task, and GraRep obtained perfect scores for the node classification task.

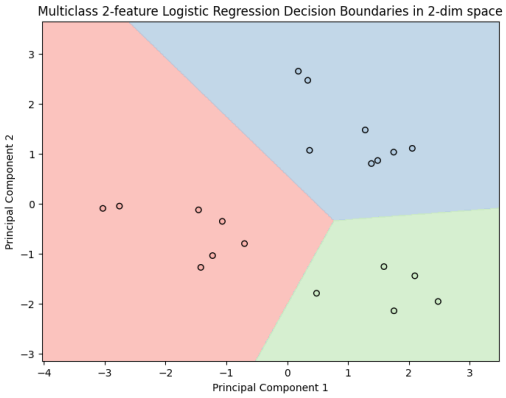
[16] This often occurs when the training set is unbalanced; i.e. contains many more data points from one class than from another



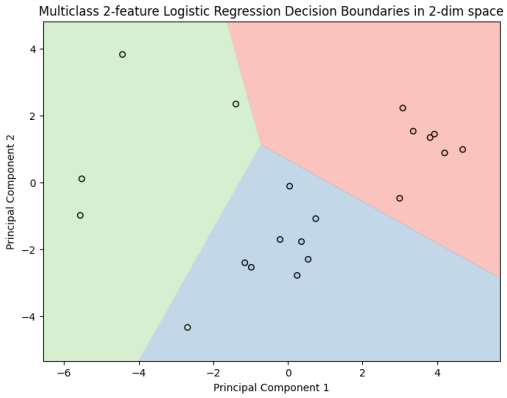
(a) Spectral Embedding



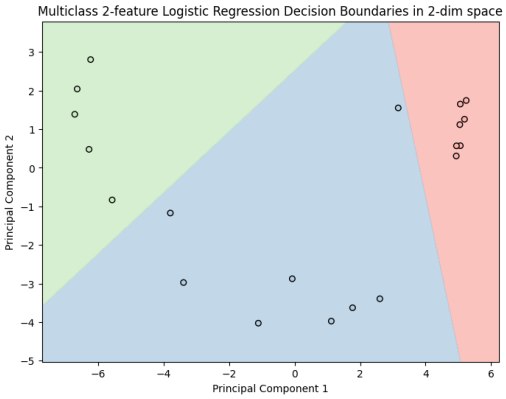
(b) Node2vec



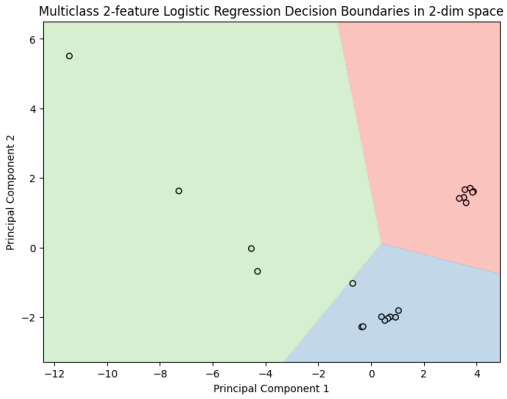
(c) LINE



(d) GraRep



(e) GraphSage



(f) DGI

Figure 3: Plots of the top two principal components for the embeddings (in the test set) produced by each embedding method. The points (embeddings) are coloured by the node's true class, and the background is shaded by the predicted class.

References

[1] Emmanuel Abbe. Community detection and stochastic block models: recent developments. *Journal of Machine Learning Research*, 18(177):1–86, 2017.

[2] D. Bilkštytė-Skanė and V. Akstinaite. Strategic organizational changes: Adopting data-driven decisions. *Strategic Change*, 33(2):107–116, 2024.

[3] Erik Brynjolfsson, Lorin M Hitt, and Heekyung H Kim. Strength in numbers: How does data-driven decisionmaking affect firm performance? *Management Science*, 57(10):1–17, 2011.

[4] H. Cai, V. W. Zheng, and K. C.-C. Chang. A comprehensive survey of network embedding: Problems, techniques, and applications. *IEEE Transactions on Knowledge and Data Engineering*, 30(9):1616–1637, 2018.

[5] Shaosheng Cao, Wei Lu, and Qiongkai Xu. Grarep: Learning graph representations with global structural information. In *Proceedings of the 24th ACM International on Conference on Information and Knowledge Management*, pages 891–900. ACM, 2015.

[6] Wei-Lin Chiang, Xuanqing Liu, Si Si, Yang Li, Samy Bengio, and Cho-Jui Hsieh. Cluster-gcn: An efficient algorithm for training deep and large graph convolutional networks. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery Data Mining*, pages 257–266. ACM, 2019.

[7] Kenneth W. Church and Patrick Hanks. Word association norms, mutual information, and lexicography. *Computational Linguistics*, 16(1):22–29, 1990.

[8] Elior Cohen. node2vec: Python implementation of the node2vec algorithm. https://github.com/eliorc/node2vec, 2017. Accessed: 2025-08-20.

[9] Michaël Defferrard, Xavier Bresson, and Pierre Vandergheynst. Convolutional neural networks on graphs with fast localized spectral filtering. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 29, pages 3844–3852, 2016.

[10] dmpierre. Line: A pytorch implementation of the line paper. https://github.com/dmpierre/LINE, 2025. Accessed: 2025-08-20.

[11] Cristian Donnat, Marinka Zitnik, Davide Hallac, and Jure Leskovec. Learning structural node embeddings via diffusion wavelets. *arXiv preprint arXiv:1710.10321*, 2018.

[12] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2017.

[13] Yoav Goldberg and Omer Levy. word2vec explained: Deriving mikolov et al.'s negative-sampling word-embedding method. *arXiv preprint arXiv:1402.3722*, 2014.

[14] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, Baltimore, MD, 4 edition, 2013.

[15] Aditya Grover and Jure Leskovec. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 855–864. ACM, 2016.

[16] Wenqi Gu, Li Yu, Jie Liang, Hanyu Jiang, Zhichao Li, Zhiyuan Li, and Shenda Liu. Graph representation learning and its applications. *Sensors*, 23(8), 2023.

[17] Michael Gutmann and Aapo Hyvärinen. Noise-contrastive estimation: A new estimation principle for unnormalized statistical models. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 297–304, 2010.

[18] William L. Hamilton. Graph representation learning. *Synthesis Lectures on Artificial Intelligence and Machine Learning*, 14(3):1–159, 2020.

[19] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[20] Paul W Holland, Kathryn Blackmond Laskey, and Samuel Leinhardt. Stochastic blockmodels: First steps. *Social Networks*, 5(2):109–137, 1983.

[21] David W. Hosmer and Stanley Lemeshow. *Applied Logistic Regression.* John Wiley & Sons, 1989.

[22] Ian T Jolliffe. *Principal Component Analysis.* Springer, 2nd edition, 2002.

[23] Ian T Jolliffe and Jorge Cadima. Principal component analysis: A review and recent developments. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 374(2065):20150202, 2016.

[24] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*, 2015.

[25] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.

[26] Yehuda Koren, Robert M Bell, and Chris J Volinsky. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37, 2009.

[27] Omer Levy and Yoav Goldberg. Neural word embedding as implicit matrix factorization. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 2177–2185, 2014.

[28] Jiah-Shin Lin. Divergence measures based on the shannon entropy. *IEEE Transactions on Information Theory*, 37(1):145–151, 1991.

[29] Rick Merritt. What are graph neural networks? NVIDIA Blog, October 2022. Online; posted October 24, 2022.

[30] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. In *Proceedings of the International Conference on Learning Representations (ICLR) Workshop*, 2013.

[31] M. E. J. Newman. *Networks: An Introduction.* Oxford University Press, Oxford, UK, 2010.

[32] Andrew Y. Ng, Michael I. Jordan, and Yair Weiss. On spectral clustering: Analysis and an algorithm. In *Advances in Neural Information Processing Systems*, volume 14, pages 849–856, 2002.

[33] Karl Pearson. On lines and planes of closest fit to systems of points in space. *Philosophical Magazine*, 2(11):559–572, 1901.

[34] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. Deepwalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 701–710. ACM, 2014.

[35] Jiezhong Qiu, Yuxiao Dong, Hao Ma, Jian Li, Kuansan Wang, and Jie Tang. Network embedding as matrix factorization: Unifying deepwalk, line, pte, and node2vec. In *Proceedings of the 11th ACM International Conference on Web Search and Data Mining (WSDM)*, pages 459–467, 2018.

[36] T. Konstantin Rusch, Michael M. Bronstein, and Siddhartha Mishra. A survey on oversmoothing in graph neural networks. *arXiv preprint arXiv:2303.10993*, 2023.

[37] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 623–656, 1948.

[38] Alexander Strehl and Joydeep Ghosh. Cluster ensembles—a knowledge reuse framework for combining multiple partitions. *Journal of Machine Learning Research*, 3:583–617, 2002.

[39] Jian Tang, Meng Qu, Mingzhe Wang, Ming Zhang, Jun Yan, and Qiaozhu Mei. Line: Large-scale information network embedding. In *Proceedings of the 24th International Conference on World Wide Web*, pages 1067–1077. ACM, 2015.

[40] C.J. van Rijsbergen. *Information Retrieval*. Butterworths, 2 edition, 1979.

[41] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.

[42] Petar Veličković, William Fedus, William L. Hamilton, Pietro Liò, Yoshua Bengio, and R. Devon Hjelm. Deep graph infomax. In *International Conference on Learning Representations (ICLR)*, 2019.

[43] Ulrike von Luxburg. A tutorial on spectral clustering. *Statistics and Computing*, 17(4):395–416, 2007.

[44] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S. Yu. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32(1):4–24, 2021.

[45] Seongjun Yun, Minbyul Jeong, Raehyun Kim, Jaewoo Kang, and Hyunwoo J. Kim. Graph transformer networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, pages 11960–11970, 2019.